JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Class LocalTime

java.lang.Object
    java.time.LocalTime

All Implemented Interfaces:

Serializable, Comparable<LocalTime>, Temporal, TemporalAccessor, TemporalAdjuster

---

```
public final class LocalTime
extends Object
implements Temporal, TemporalAdjuster, Comparable<LocalTime>,
Serializable
```

A time without a time-zone in the ISO-8601 calendar system, such as `10:15:30`.

`LocalTime` is an immutable date-time object that represents a time, often viewed as hour-minute-second. Time is represented to nanosecond precision. For example, the value "13:45.30.123456789" can be stored in a `LocalTime`.

This class does not store or represent a date or time-zone. Instead, it is a description of the local time as seen on a wall clock. It cannot represent an instant on the time-line without additional information such as an offset or time-zone.

The ISO-8601 calendar system is the modern civil calendar system used today in most of the world. This API assumes that all calendar systems use the same representation, this class, for time-of-day.

This is a value-based class; programmers should treat instances that are equal as interchangeable and should not use instances for synchronization, or unpredictable behavior may occur. For example, in a future release, synchronization may fail. The `equals` method should be used for comparisons.

Implementation Requirements:

This class is immutable and thread-safe.

Since:

1.8

See Also:

Serialized Form

Field Summary

Fields

Modifier and Type	Field	Description

static final LocalTime	MAX	The maximum supported LocalTime, '23:59:59.999999999'.
static final LocalTime	MIDNIGHT	The time of midnight at the start of the day, '00:00'.
static final LocalTime	MIN	The minimum supported LocalTime, '00:00'.
static final LocalTime	NOON	The time of noon in the middle of the day, '12:00'.

Method Summary

All Methods | Static Methods

Modifier and Type	Method	Description
Temporal	adjustInto(Temporal temporal)	Adjusts the specified temporal object to have the same time as this object.
LocalDateTime	atDate(LocalDate date)	Combines this time with a date to create a LocalDateTime.
OffsetTime	atOffset(ZoneOffset offset)	Combines this time with an offset to create an OffsetTime.
int	compareTo(LocalTime other)	Compares this time to another time.
boolean	equals(Object obj)	Checks if this time is equal to another time.
String	format(DateTimeFormatter forma	Formats this time using the specified formatter.
static LocalTime	from(TemporalAccessor tempor	

		Obtains an instance of `LocalTime` from a temporal object.
`int`	**`get`**`(TemporalField field)`	Gets the value of the specified field from this time as an `int`.
`int`	**`getHour`**`()`	Gets the hour-of-day field.
`long`	**`getLong`**`(TemporalField field)`	Gets the value of the specified field from this time as a `long`.
`int`	**`getMinute`**`()`	Gets the minute-of-hour field.
`int`	**`getNano`**`()`	Gets the nano-of-second field.
`int`	**`getSecond`**`()`	Gets the second-of-minute field.
`int`	**`hashCode`**`()`	A hash code for this time.
`boolean`	**`isAfter`**`(LocalTime other)`	Checks if this time is after the specified time.
`boolean`	**`isBefore`**`(LocalTime other)`	Checks if this time is before the specified time.
`boolean`	**`isSupported`**`(TemporalField field)`	Checks if the specified field is supported.
`boolean`	**`isSupported`**`(TemporalUnit unit)`	Checks if the specified unit is supported.
`LocalTime`	**`minus`**`(long amountToSubtract, TemporalUnit unit)`	Returns a copy of this time with the specified amount subtracted.
`LocalTime`	**`minus`**`(TemporalAmount amountTo`	Returns a copy of this time with the specified amount subtracted.

LocalTime	**minusHours**(long hoursToSubtract)	Returns a copy of this LocalTime with the specified number of hours subtracted.
LocalTime	**minusMinutes**(long minutesToSubtract)	Returns a copy of this LocalTime with the specified number of minutes subtracted.
LocalTime	**minusNanos**(long nanosToSubtract)	Returns a copy of this LocalTime with the specified number of nanoseconds subtracted.
LocalTime	**minusSeconds**(long secondsToSubtract)	Returns a copy of this LocalTime with the specified number of seconds subtracted.
static LocalTime	**now**()	Obtains the current time from the system clock in the default time-zone.
static LocalTime	**now**(Clock clock)	Obtains the current time from the specified clock.
static LocalTime	**now**(ZoneId zone)	Obtains the current time from the system clock in the specified time-zone.
static LocalTime	**of**(int hour, int minute)	Obtains an instance of LocalTime from an hour and minute.
static LocalTime	**of**(int hour, int minute, int second)	Obtains an instance of LocalTime from an hour, minute and second.
static LocalTime	**of**(int hour, int minute, int second, int nanoOfSecond)	Obtains an instance of LocalTime from an hour, minute, second and nanosecond.
static LocalTime		Obtains an instance of LocalTime from an Instant and zone ID.

	ofInstant (Instant instant, ZoneId zone)	
static LocalTime	**ofNanoOfDay** (long nanoOfDay)	Obtains an instance of LocalTime from a nanos-of-day value.
static LocalTime	**ofSecondOfDay** (long secondOfDay)	Obtains an instance of LocalTime from a second-of-day value.
static LocalTime	**parse** (CharSequence text)	Obtains an instance of LocalTime from a text string such as 10:15.
static LocalTime	**parse** (CharSequence text, DateTimeFormatter format	Obtains an instance of LocalTime from a text string using a specific formatter.
LocalTime	**plus**(long amountToAdd, TemporalUnit unit)	Returns a copy of this time with the specified amount added.
LocalTime	**plus** (TemporalAmount amountTo	Returns a copy of this time with the specified amount added.
LocalTime	**plusHours** (long hoursToAdd)	Returns a copy of this LocalTime with the specified number of hours added.
LocalTime	**plusMinutes** (long minutesToAdd)	Returns a copy of this LocalTime with the specified number of minutes added.
LocalTime	**plusNanos** (long nanosToAdd)	Returns a copy of this LocalTime with the specified number of nanoseconds added.
LocalTime	**plusSeconds** (long secondstoAdd)	Returns a copy of this LocalTime with the

		specified number of seconds added.
<R> R	**query** (TemporalQuery<R> query)	Queries this time using the specified query.
ValueRange	**range** (TemporalField field)	Gets the range of valid values for the specified field.
long	**toEpochSecond** (LocalDate date, ZoneOffset offset)	Converts this LocalTime to the number of seconds since the epoch of 1970-01-01T00:00:00Z.
long	**toNanoOfDay**()	Extracts the time as nanos of day, from 0 to 24 * 60 * 60 * 1,000,000,000 - 1.
int	**toSecondOfDay**()	Extracts the time as seconds of day, from 0 to 24 * 60 * 60 - 1.
String	**toString**()	Outputs this time as a String, such as 10:15.
LocalTime	**truncatedTo** (TemporalUnit unit)	Returns a copy of this LocalTime with the time truncated.
long	**until** (Temporal endExclusive, TemporalUnit unit)	Calculates the amount of time until another time in terms of the specified unit.
LocalTime	**with** (TemporalAdjuster adjust	Returns an adjusted copy of this time.
LocalTime	**with** (TemporalField field, long newValue)	Returns a copy of this time with the specified field set to a new value.
LocalTime	**withHour**(int hour)	

		Returns a copy of this LocalTime with the hour-of-day altered.
LocalTime	**withMinute**(int minute)	Returns a copy of this LocalTime with the minute-of-hour altered.
LocalTime	**withNano**(int nanoOfSecond)	Returns a copy of this LocalTime with the nano-of-second altered.
LocalTime	**withSecond**(int second)	Returns a copy of this LocalTime with the second-of-minute altered.

Methods declared in class Object

clone, finalize, getClass, notify, notifyAll, wait, wait, wait

Field Details

MIN

```
public static final LocalTime MIN
```

The minimum supported LocalTime, '00:00'. This is the time of midnight at the start of the day.

MAX

```
public static final LocalTime MAX
```

The maximum supported LocalTime, '23:59:59.999999999'. This is the time just before midnight at the end of the day.

MIDNIGHT

```
public static final LocalTime MIDNIGHT
```

The time of midnight at the start of the day, '00:00'.

NOON

```
public static final LocalTime NOON
```

The time of noon in the middle of the day, '12:00'.

Method Details

now

`public static LocalTime now()`

Obtains the current time from the system clock in the default time-zone.

This will query the `system clock` in the default time-zone to obtain the current time.

Using this method will prevent the ability to use an alternate clock for testing because the clock is hard-coded.

Returns:

the current time using the system clock and default time-zone, not null

now

`public static LocalTime now(ZoneId zone)`

Obtains the current time from the system clock in the specified time-zone.

This will query the `system clock` to obtain the current time. Specifying the time-zone avoids dependence on the default time-zone.

Using this method will prevent the ability to use an alternate clock for testing because the clock is hard-coded.

Parameters:

`zone` - the zone ID to use, not null

Returns:

the current time using the system clock, not null

now

`public static LocalTime now(Clock clock)`

Obtains the current time from the specified clock.

This will query the specified clock to obtain the current time. Using this method allows the use of an alternate clock for testing. The alternate clock may be introduced using `dependency injection`.

Parameters:

`clock` - the clock to use, not null

Returns:

the current time, not null

of

```
public static LocalTime of(int hour,
                           int minute)
```

Obtains an instance of `LocalTime` from an hour and minute.

This returns a `LocalTime` with the specified hour and minute. The second and nanosecond fields will be set to zero.

Parameters:

`hour` - the hour-of-day to represent, from 0 to 23

`minute` - the minute-of-hour to represent, from 0 to 59

Returns:

the local time, not null

Throws:

`DateTimeException` - if the value of any field is out of range

of

```
public static LocalTime of(int hour,
                           int minute,
                           int second)
```

Obtains an instance of `LocalTime` from an hour, minute and second.

This returns a `LocalTime` with the specified hour, minute and second. The nanosecond field will be set to zero.

Parameters:

`hour` - the hour-of-day to represent, from 0 to 23

`minute` - the minute-of-hour to represent, from 0 to 59

`second` - the second-of-minute to represent, from 0 to 59

Returns:

the local time, not null

Throws:

`DateTimeException` - if the value of any field is out of range

of

```
public static LocalTime of(int hour,
                           int minute,
                           int second,
                           int nanoOfSecond)
```

Obtains an instance of `LocalTime` from an hour, minute, second and nanosecond.

This returns a `LocalTime` with the specified hour, minute, second and nanosecond.

Parameters:

`hour` - the hour-of-day to represent, from 0 to 23

`minute` - the minute-of-hour to represent, from 0 to 59

`second` - the second-of-minute to represent, from 0 to 59

`nanoOfSecond` - the nano-of-second to represent, from 0 to 999,999,999

Returns:

the local time, not null

Throws:

`DateTimeException` - if the value of any field is out of range

ofInstant

```
public static LocalTime ofInstant(Instant instant,
                                  ZoneId zone)
```

Obtains an instance of `LocalTime` from an `Instant` and zone ID.

This creates a local time based on the specified instant. First, the offset from UTC/Greenwich is obtained using the zone ID and instant, which is simple as there is only one valid offset for each instant. Then, the instant and offset are used to calculate the local time.

Parameters:

`instant` - the instant to create the time from, not null

`zone` - the time-zone, which may be an offset, not null

Returns:

the local time, not null

Since:

9

ofSecondOfDay

```
public static LocalTime ofSecondOfDay(long secondOfDay)
```

Obtains an instance of `LocalTime` from a second-of-day value.

This returns a `LocalTime` with the specified second-of-day. The nanosecond field will be set to zero.

Parameters:

`secondOfDay` - the second-of-day, from `0` to `24 * 60 * 60 - 1`

Returns:

the local time, not null

Throws:

`DateTimeException` - if the second-of-day value is invalid

ofNanoOfDay

```
public static LocalTime ofNanoOfDay(long nanoOfDay)
```

Obtains an instance of `LocalTime` from a nanos-of-day value.

This returns a `LocalTime` with the specified nanosecond-of-day.

Parameters:

`nanoOfDay` - the nano of day, from `0` to `24 * 60 * 60 * 1,000,000,000 - 1`

Returns:

the local time, not null

Throws:

`DateTimeException` - if the nanos of day value is invalid

from

```
public static LocalTime from(TemporalAccessor temporal)
```

Obtains an instance of `LocalTime` from a temporal object.

This obtains a local time based on the specified temporal. A `TemporalAccessor` represents an arbitrary set of date and time information, which this factory converts to an instance of `LocalTime`.

The conversion uses the `TemporalQueries.localTime()` query, which relies on extracting the `NANO_OF_DAY` field.

This method matches the signature of the functional interface `TemporalQuery` allowing it to be used as a query via method reference, `LocalTime::from`.

Parameters:

`temporal` - the temporal object to convert, not null

Returns:

the local time, not null

Throws:

`DateTimeException` - if unable to convert to a `LocalTime`

parse

```
public static LocalTime parse(CharSequence text)
```

Obtains an instance of `LocalTime` from a text string such as `10:15`.

The string must represent a valid time and is parsed using `DateTimeFormatter.ISO_LOCAL_TIME`.

Parameters:

`text` - the text to parse such as "10:15:30", not null

Returns:

the parsed local time, not null

Throws:

`DateTimeParseException` - if the text cannot be parsed

parse

```
public static LocalTime parse(CharSequence text,
                              DateTimeFormatter formatter)
```

Obtains an instance of `LocalTime` from a text string using a specific formatter.

The text is parsed using the formatter, returning a time.

Parameters:

`text` - the text to parse, not null

`formatter` - the formatter to use, not null

Returns:

the parsed local time, not null

Throws:

`DateTimeParseException` - if the text cannot be parsed

isSupported

```
public boolean isSupported(TemporalField field)
```

Checks if the specified field is supported.

This checks if this time can be queried for the specified field. If false, then calling the `range`, `get` and `with(TemporalField, long)` methods will throw an exception.

If the field is a `ChronoField` then the query is implemented here. The supported fields are:

- `NANO_OF_SECOND`
- `NANO_OF_DAY`
- `MICRO_OF_SECOND`
- `MICRO_OF_DAY`
- `MILLI_OF_SECOND`
- `MILLI_OF_DAY`
- `SECOND_OF_MINUTE`
- `SECOND_OF_DAY`
- `MINUTE_OF_HOUR`
- `MINUTE_OF_DAY`
- `HOUR_OF_AMPM`
- `CLOCK_HOUR_OF_AMPM`
- `HOUR_OF_DAY`
- `CLOCK_HOUR_OF_DAY`
- `AMPM_OF_DAY`

All other `ChronoField` instances will return false.

If the field is not a `ChronoField`, then the result of this method is obtained by invoking `TemporalField.isSupportedBy(TemporalAccessor)` passing `this` as the argument. Whether the field is supported is determined by the field.

Specified by:

`isSupported` in interface `TemporalAccessor`

Parameters:

`field` - the field to check, null returns false

Returns:

true if the field is supported on this time, false if not

isSupported

```
public boolean isSupported(TemporalUnit unit)
```

Checks if the specified unit is supported.

This checks if the specified unit can be added to, or subtracted from, this time. If false, then calling the `plus(long, TemporalUnit)` and `minus` methods will throw an exception.

If the unit is a `ChronoUnit` then the query is implemented here. The supported units are:

- `NANOS`
- `MICROS`
- `MILLIS`
- `SECONDS`
- `MINUTES`
- `HOURS`
- `HALF_DAYS`

All other `ChronoUnit` instances will return false.

If the unit is not a `ChronoUnit`, then the result of this method is obtained by invoking `TemporalUnit.isSupportedBy(Temporal)` passing `this` as the argument. Whether the unit is supported is determined by the unit.

Specified by:

`isSupported` in interface `Temporal`

Parameters:

`unit` - the unit to check, null returns false

Returns:

true if the unit can be added/subtracted, false if not

range

`public ValueRange range(TemporalField field)`

Gets the range of valid values for the specified field.

The range object expresses the minimum and maximum valid values for a field. This time is used to enhance the accuracy of the returned range. If it is not possible to return the range, because the field is not supported or for some other reason, an exception is thrown.

If the field is a `ChronoField` then the query is implemented here. The `supported fields` will return appropriate range instances. All other `ChronoField` instances will throw an `UnsupportedTemporalTypeException`.

If the field is not a `ChronoField`, then the result of this method is obtained by invoking `TemporalField.rangeRefinedBy(TemporalAccessor)` passing `this` as the argument. Whether the range can be obtained is determined by the field.

Specified by:

`range` in interface `TemporalAccessor`

Parameters:

`field` - the field to query the range for, not null

Returns:

the range of valid values for the field, not null

Throws:

`DateTimeException` - if the range for the field cannot be obtained

`UnsupportedTemporalTypeException` - if the field is not supported

get

```
public int get(TemporalField field)
```

Gets the value of the specified field from this time as an `int`.

This queries this time for the value of the specified field. The returned value will always be within the valid range of values for the field. If it is not possible to return the value, because the field is not supported or for some other reason, an exception is thrown.

If the field is a `ChronoField` then the query is implemented here. The `supported fields` will return valid values based on this time, except `NANO_OF_DAY` and `MICRO_OF_DAY` which are too large to fit in an `int` and throw an `UnsupportedTemporalTypeException`. All other `ChronoField` instances will throw an `UnsupportedTemporalTypeException`.

If the field is not a `ChronoField`, then the result of this method is obtained by invoking `TemporalField.getFrom(TemporalAccessor)` passing `this` as the argument. Whether the value can be obtained, and what the value represents, is determined by the field.

Specified by:

`get` in interface `TemporalAccessor`

Parameters:

`field` - the field to get, not null

Returns:

the value for the field

Throws:

`DateTimeException` - if a value for the field cannot be obtained or the value is outside the range of valid values for the field

`UnsupportedTemporalTypeException` - if the field is not supported or the range of values exceeds an `int`

`ArithmeticException` - if numeric overflow occurs

getLong

```
public long getLong(TemporalField field)
```

Gets the value of the specified field from this time as a `long`.

This queries this time for the value of the specified field. If it is not possible to return the value, because the field is not supported or for some other reason, an exception is thrown.

If the field is a `ChronoField` then the query is implemented here. The `supported fields` will return valid values based on this time. All other `ChronoField` instances will throw an `UnsupportedTemporalTypeException`.

If the field is not a `ChronoField`, then the result of this method is obtained by invoking `TemporalField.getFrom(TemporalAccessor)` passing `this` as the argument. Whether the value can be obtained, and what the value represents, is determined by the field.

Specified by:

`getLong` in interface `TemporalAccessor`

Parameters:

`field` - the field to get, not null

Returns:

the value for the field

Throws:

`DateTimeException` - if a value for the field cannot be obtained

`UnsupportedTemporalTypeException` - if the field is not supported

`ArithmeticException` - if numeric overflow occurs

getHour

```
public int getHour()
```

Gets the hour-of-day field.

Returns:

the hour-of-day, from 0 to 23

getMinute

```
public int getMinute()
```

Gets the minute-of-hour field.

Returns:

the minute-of-hour, from 0 to 59

getSecond

```
public int getSecond()
```

Gets the second-of-minute field.

Returns:

the second-of-minute, from 0 to 59

getNano

```
public int getNano()
```

Gets the nano-of-second field.

Returns:

the nano-of-second, from 0 to 999,999,999

with

```
public LocalTime with(TemporalAdjuster adjuster)
```

Returns an adjusted copy of this time.

This returns a `LocalTime`, based on this one, with the time adjusted. The adjustment takes place using the specified adjuster strategy object. Read the documentation of the adjuster to understand what adjustment will be made.

A simple adjuster might simply set the one of the fields, such as the hour field. A more complex adjuster might set the time to the last hour of the day.

The result of this method is obtained by invoking the `TemporalAdjuster.adjustInto(Temporal)` method on the specified adjuster passing `this` as the argument.

This instance is immutable and unaffected by this method call.

Specified by:

`with` in interface `Temporal`

Parameters:

`adjuster` - the adjuster to use, not null

Returns:

a `LocalTime` based on `this` with the adjustment made, not null

Throws:

`DateTimeException` - if the adjustment cannot be made

`ArithmeticException` - if numeric overflow occurs

with

```
public LocalTime with(TemporalField field,
                      long newValue)
```

Returns a copy of this time with the specified field set to a new value.

This returns a `LocalTime`, based on this one, with the value for the specified field changed. This can be used to change any supported field, such as the hour, minute or second. If it is not possible to set the value, because the field is not supported or for some other reason, an exception is thrown.

If the field is a `ChronoField` then the adjustment is implemented here. The supported fields behave as follows:

- `NANO_OF_SECOND` - Returns a `LocalTime` with the specified nano-of-second. The hour, minute and second will be unchanged.
- `NANO_OF_DAY` - Returns a `LocalTime` with the specified nano-of-day. This completely replaces the time and is equivalent to `ofNanoOfDay(long)`.
- `MICRO_OF_SECOND` - Returns a `LocalTime` with the nano-of-second replaced by the specified micro-of-second multiplied by 1,000. The hour, minute and second will be unchanged.
- `MICRO_OF_DAY` - Returns a `LocalTime` with the specified micro-of-day. This completely replaces the time and is equivalent to using `ofNanoOfDay(long)` with the micro-of-day multiplied by 1,000.
- `MILLI_OF_SECOND` - Returns a `LocalTime` with the nano-of-second replaced by the specified milli-of-second multiplied by 1,000,000. The hour, minute and second will be unchanged.
- `MILLI_OF_DAY` - Returns a `LocalTime` with the specified milli-of-day. This completely replaces the time and is equivalent to using `ofNanoOfDay(long)` with the milli-of-day multiplied by 1,000,000.
- `SECOND_OF_MINUTE` - Returns a `LocalTime` with the specified second-of-minute. The hour, minute and nano-of-second will be unchanged.
- `SECOND_OF_DAY` - Returns a `LocalTime` with the specified second-of-day. The nano-of-second will be unchanged.
- `MINUTE_OF_HOUR` - Returns a `LocalTime` with the specified minute-of-hour. The hour, second-of-minute and nano-of-second will be unchanged.

- `MINUTE_OF_DAY` - Returns a `LocalTime` with the specified minute-of-day. The second-of-minute and nano-of-second will be unchanged.
- `HOUR_OF_AMPM` - Returns a `LocalTime` with the specified hour-of-am-pm. The AM/PM, minute-of-hour, second-of-minute and nano-of-second will be unchanged.
- `CLOCK_HOUR_OF_AMPM` - Returns a `LocalTime` with the specified clock-hour-of-am-pm. The AM/PM, minute-of-hour, second-of-minute and nano-of-second will be unchanged.
- `HOUR_OF_DAY` - Returns a `LocalTime` with the specified hour-of-day. The minute-of-hour, second-of-minute and nano-of-second will be unchanged.
- `CLOCK_HOUR_OF_DAY` - Returns a `LocalTime` with the specified clock-hour-of-day. The minute-of-hour, second-of-minute and nano-of-second will be unchanged.
- `AMPM_OF_DAY` - Returns a `LocalTime` with the specified AM/PM. The hour-of-am-pm, minute-of-hour, second-of-minute and nano-of-second will be unchanged.

In all cases, if the new value is outside the valid range of values for the field then a `DateTimeException` will be thrown.

All other `ChronoField` instances will throw an `UnsupportedTemporalTypeException`.

If the field is not a `ChronoField`, then the result of this method is obtained by invoking `TemporalField.adjustInto(Temporal, long)` passing `this` as the argument. In this case, the field determines whether and how to adjust the instant.

This instance is immutable and unaffected by this method call.

Specified by:

`with` in interface `Temporal`

Parameters:

`field` - the field to set in the result, not null

`newValue` - the new value of the field in the result

Returns:

a `LocalTime` based on `this` with the specified field set, not null

Throws:

`DateTimeException` - if the field cannot be set

`UnsupportedTemporalTypeException` - if the field is not supported

`ArithmeticException` - if numeric overflow occurs

withHour

`public LocalTime withHour(int hour)`

Returns a copy of this `LocalTime` with the hour-of-day altered.

This instance is immutable and unaffected by this method call.

Parameters:

`hour` - the hour-of-day to set in the result, from 0 to 23

Returns:

a `LocalTime` based on this time with the requested hour, not null

Throws:

`DateTimeException` - if the hour value is invalid

withMinute

`public LocalTime withMinute(int minute)`

Returns a copy of this `LocalTime` with the minute-of-hour altered.

This instance is immutable and unaffected by this method call.

Parameters:

`minute` - the minute-of-hour to set in the result, from 0 to 59

Returns:

a `LocalTime` based on this time with the requested minute, not null

Throws:

`DateTimeException` - if the minute value is invalid

withSecond

`public LocalTime withSecond(int second)`

Returns a copy of this `LocalTime` with the second-of-minute altered.

This instance is immutable and unaffected by this method call.

Parameters:

`second` - the second-of-minute to set in the result, from 0 to 59

Returns:

a `LocalTime` based on this time with the requested second, not null

Throws:

`DateTimeException` - if the second value is invalid

withNano

`public LocalTime withNano(int nanoOfSecond)`

Returns a copy of this `LocalTime` with the nano-of-second altered.

This instance is immutable and unaffected by this method call.

Parameters:

`nanoOfSecond` - the nano-of-second to set in the result, from 0 to 999,999,999

Returns:

a `LocalTime` based on this time with the requested nanosecond, not null

Throws:

`DateTimeException` - if the nanos value is invalid

truncatedTo

`public LocalTime truncatedTo(TemporalUnit unit)`

Returns a copy of this `LocalTime` with the time truncated.

Truncation returns a copy of the original time with fields smaller than the specified unit set to zero. For example, truncating with the `minutes` unit will set the second-of-minute and nano-of-second field to zero.

The unit must have a duration that divides into the length of a standard day without remainder. This includes all supplied time units on `ChronoUnit` and `DAYS`. Other units throw an exception.

This instance is immutable and unaffected by this method call.

Parameters:

`unit` - the unit to truncate to, not null

Returns:

a `LocalTime` based on this time with the time truncated, not null

Throws:

`DateTimeException` - if unable to truncate

`UnsupportedTemporalTypeException` - if the unit is not supported

plus

`public LocalTime plus(TemporalAmount amountToAdd)`

Returns a copy of this time with the specified amount added.

This returns a `LocalTime`, based on this one, with the specified amount added. The amount is typically `Duration` but may be any other type implementing the `TemporalAmount` interface.

The calculation is delegated to the amount object by calling `TemporalAmount.addTo(Temporal)`. The amount implementation is free to implement the addition in any way it wishes, however it typically calls back to `plus(long, TemporalUnit)`. Consult the documentation of the amount implementation to determine if it can be successfully added.

This instance is immutable and unaffected by this method call.

Specified by:

`plus` in interface `Temporal`

Parameters:

`amountToAdd` - the amount to add, not null

Returns:

a `LocalTime` based on this time with the addition made, not null

Throws:

`DateTimeException` - if the addition cannot be made

`ArithmeticException` - if numeric overflow occurs

plus

```
public LocalTime plus(long amountToAdd,
                      TemporalUnit unit)
```

Returns a copy of this time with the specified amount added.

This returns a `LocalTime`, based on this one, with the amount in terms of the unit added. If it is not possible to add the amount, because the unit is not supported or for some other reason, an exception is thrown.

If the field is a `ChronoUnit` then the addition is implemented here. The supported fields behave as follows:

- `NANOS` - Returns a `LocalTime` with the specified number of nanoseconds added. This is equivalent to `plusNanos(long)`.
- `MICROS` - Returns a `LocalTime` with the specified number of microseconds added. This is equivalent to `plusNanos(long)` with the amount multiplied by 1,000.
- `MILLIS` - Returns a `LocalTime` with the specified number of milliseconds added. This is equivalent to `plusNanos(long)` with the amount multiplied by 1,000,000.

- `SECONDS` - Returns a `LocalTime` with the specified number of seconds added. This is equivalent to `plusSeconds(long)`.
- `MINUTES` - Returns a `LocalTime` with the specified number of minutes added. This is equivalent to `plusMinutes(long)`.
- `HOURS` - Returns a `LocalTime` with the specified number of hours added. This is equivalent to `plusHours(long)`.
- `HALF_DAYS` - Returns a `LocalTime` with the specified number of half-days added. This is equivalent to `plusHours(long)` with the amount multiplied by 12.

All other `ChronoUnit` instances will throw an `UnsupportedTemporalTypeException`.

If the field is not a `ChronoUnit`, then the result of this method is obtained by invoking `TemporalUnit.addTo(Temporal, long)` passing `this` as the argument. In this case, the unit determines whether and how to perform the addition.

This instance is immutable and unaffected by this method call.

Specified by:

`plus` in interface `Temporal`

Parameters:

`amountToAdd` - the amount of the unit to add to the result, may be negative

`unit` - the unit of the amount to add, not null

Returns:

a `LocalTime` based on this time with the specified amount added, not null

Throws:

`DateTimeException` - if the addition cannot be made

`UnsupportedTemporalTypeException` - if the unit is not supported

`ArithmeticException` - if numeric overflow occurs

plusHours

```
public LocalTime plusHours(long hoursToAdd)
```

Returns a copy of this `LocalTime` with the specified number of hours added.

This adds the specified number of hours to this time, returning a new time. The calculation wraps around midnight.

This instance is immutable and unaffected by this method call.

Parameters:

`hoursToAdd` - the hours to add, may be negative

Returns:

a `LocalTime` based on this time with the hours added, not null

plusMinutes

```
public LocalTime plusMinutes(long minutesToAdd)
```

Returns a copy of this `LocalTime` with the specified number of minutes added.

This adds the specified number of minutes to this time, returning a new time. The calculation wraps around midnight.

This instance is immutable and unaffected by this method call.

Parameters:

`minutesToAdd` - the minutes to add, may be negative

Returns:

a `LocalTime` based on this time with the minutes added, not null

plusSeconds

```
public LocalTime plusSeconds(long secondstoAdd)
```

Returns a copy of this `LocalTime` with the specified number of seconds added.

This adds the specified number of seconds to this time, returning a new time. The calculation wraps around midnight.

This instance is immutable and unaffected by this method call.

Parameters:

`secondstoAdd` - the seconds to add, may be negative

Returns:

a `LocalTime` based on this time with the seconds added, not null

plusNanos

```
public LocalTime plusNanos(long nanosToAdd)
```

Returns a copy of this `LocalTime` with the specified number of nanoseconds added.

This adds the specified number of nanoseconds to this time, returning a new time. The calculation wraps around midnight.

This instance is immutable and unaffected by this method call.

Parameters:

`nanosToAdd` - the nanos to add, may be negative

Returns:

a `LocalTime` based on this time with the nanoseconds added, not null

minus

```
public LocalTime minus(TemporalAmount amountToSubtract)
```

Returns a copy of this time with the specified amount subtracted.

This returns a `LocalTime`, based on this one, with the specified amount subtracted. The amount is typically `Duration` but may be any other type implementing the `TemporalAmount` interface.

The calculation is delegated to the amount object by calling `TemporalAmount.subtractFrom(Temporal)`. The amount implementation is free to implement the subtraction in any way it wishes, however it typically calls back to `minus(long, TemporalUnit)`. Consult the documentation of the amount implementation to determine if it can be successfully subtracted.

This instance is immutable and unaffected by this method call.

Specified by:

`minus` in interface `Temporal`

Parameters:

`amountToSubtract` - the amount to subtract, not null

Returns:

a `LocalTime` based on this time with the subtraction made, not null

Throws:

`DateTimeException` - if the subtraction cannot be made

`ArithmeticException` - if numeric overflow occurs

minus

```
public LocalTime minus(long amountToSubtract,
                       TemporalUnit unit)
```

Returns a copy of this time with the specified amount subtracted.

This returns a `LocalTime`, based on this one, with the amount in terms of the unit subtracted. If it is not possible to subtract the amount, because the unit is not supported or for some other reason, an exception is thrown.

This method is equivalent to `plus(long, TemporalUnit)` with the amount negated. See that method for a full description of how addition, and thus subtraction, works.

This instance is immutable and unaffected by this method call.

Specified by:

`minus` in interface `Temporal`

Parameters:

`amountToSubtract` - the amount of the unit to subtract from the result, may be negative

`unit` - the unit of the amount to subtract, not null

Returns:

a `LocalTime` based on this time with the specified amount subtracted, not null

Throws:

`DateTimeException` - if the subtraction cannot be made

`UnsupportedTemporalTypeException` - if the unit is not supported

`ArithmeticException` - if numeric overflow occurs

minusHours

```
public LocalTime minusHours(long hoursToSubtract)
```

Returns a copy of this `LocalTime` with the specified number of hours subtracted.

This subtracts the specified number of hours from this time, returning a new time. The calculation wraps around midnight.

This instance is immutable and unaffected by this method call.

Parameters:

`hoursToSubtract` - the hours to subtract, may be negative

Returns:

a `LocalTime` based on this time with the hours subtracted, not null

minusMinutes

```
public LocalTime minusMinutes(long minutesToSubtract)
```

Returns a copy of this `LocalTime` with the specified number of minutes subtracted.

This subtracts the specified number of minutes from this time, returning a new time. The calculation wraps around midnight.

This instance is immutable and unaffected by this method call.

Parameters:

`minutesToSubtract` - the minutes to subtract, may be negative

Returns:

a `LocalTime` based on this time with the minutes subtracted, not null

minusSeconds

`public LocalTime minusSeconds(long secondsToSubtract)`

Returns a copy of this `LocalTime` with the specified number of seconds subtracted.

This subtracts the specified number of seconds from this time, returning a new time. The calculation wraps around midnight.

This instance is immutable and unaffected by this method call.

Parameters:

`secondsToSubtract` - the seconds to subtract, may be negative

Returns:

a `LocalTime` based on this time with the seconds subtracted, not null

minusNanos

`public LocalTime minusNanos(long nanosToSubtract)`

Returns a copy of this `LocalTime` with the specified number of nanoseconds subtracted.

This subtracts the specified number of nanoseconds from this time, returning a new time. The calculation wraps around midnight.

This instance is immutable and unaffected by this method call.

Parameters:

`nanosToSubtract` - the nanos to subtract, may be negative

Returns:

a `LocalTime` based on this time with the nanoseconds subtracted, not null

query

`public <R> R query(TemporalQuery<R> query)`

Queries this time using the specified query.

This queries this time using the specified query strategy object. The `TemporalQuery` object defines the logic to be used to obtain the result. Read the documentation of the query to understand what the result of this method will be.

The result of this method is obtained by invoking the `TemporalQuery.queryFrom(TemporalAccessor)` method on the specified query passing `this` as the argument.

Specified by:

`query` in interface `TemporalAccessor`

Type Parameters:

R - the type of the result

Parameters:

query - the query to invoke, not null

Returns:

the query result, null may be returned (defined by the query)

Throws:

`DateTimeException` - if unable to query (defined by the query)

`ArithmeticException` - if numeric overflow occurs (defined by the query)

adjustInto

```
public Temporal adjustInto(Temporal temporal)
```

Adjusts the specified temporal object to have the same time as this object.

This returns a temporal object of the same observable type as the input with the time changed to be the same as this.

The adjustment is equivalent to using `Temporal.with(TemporalField, long)` passing `ChronoField.NANO_OF_DAY` as the field.

In most cases, it is clearer to reverse the calling pattern by using `Temporal.with(TemporalAdjuster)`:

```
// these two lines are equivalent, but the second approach is recommended
temporal = thisLocalTime.adjustInto(temporal);
temporal = temporal.with(thisLocalTime);
```

This instance is immutable and unaffected by this method call.

Specified by:

adjustInto in interface TemporalAdjuster

Parameters:

temporal - the target object to be adjusted, not null

Returns:

the adjusted object, not null

Throws:

DateTimeException - if unable to make the adjustment

ArithmeticException - if numeric overflow occurs

until

```
public long until(Temporal endExclusive,
                  TemporalUnit unit)
```

Calculates the amount of time until another time in terms of the specified unit.

This calculates the amount of time between two LocalTime objects in terms of a single TemporalUnit. The start and end points are this and the specified time. The result will be negative if the end is before the start. The Temporal passed to this method is converted to a LocalTime using from(TemporalAccessor). For example, the amount in hours between two times can be calculated using startTime.until(endTime, HOURS).

The calculation returns a whole number, representing the number of complete units between the two times. For example, the amount in hours between 11:30 and 13:29 will only be one hour as it is one minute short of two hours.

There are two equivalent ways of using this method. The first is to invoke this method. The second is to use TemporalUnit.between(Temporal, Temporal):

```
// these two lines are equivalent
amount = start.until(end, MINUTES);
amount = MINUTES.between(start, end);
```

The choice should be made based on which makes the code more readable.

The calculation is implemented in this method for ChronoUnit. The units NANOS, MICROS, MILLIS, SECONDS, MINUTES, HOURS and HALF_DAYS are supported. Other ChronoUnit values will throw an exception.

If the unit is not a ChronoUnit, then the result of this method is obtained by invoking TemporalUnit.between(Temporal, Temporal) passing this as the first argument and the converted input temporal as the second argument.

This instance is immutable and unaffected by this method call.

Specified by:

`until` in interface `Temporal`

Parameters:

`endExclusive` - the end time, exclusive, which is converted to a `LocalTime`, not null

`unit` - the unit to measure the amount in, not null

Returns:

the amount of time between this time and the end time

Throws:

`DateTimeException` - if the amount cannot be calculated, or the end temporal cannot be converted to a `LocalTime`

`UnsupportedTemporalTypeException` - if the unit is not supported

`ArithmeticException` - if numeric overflow occurs

format

`public String format(DateTimeFormatter formatter)`

Formats this time using the specified formatter.

This time will be passed to the formatter to produce a string.

Parameters:

`formatter` - the formatter to use, not null

Returns:

the formatted time string, not null

Throws:

`DateTimeException` - if an error occurs during printing

atDate

`public LocalDateTime atDate(LocalDate date)`

Combines this time with a date to create a `LocalDateTime`.

This returns a `LocalDateTime` formed from this time at the specified date. All possible combinations of date and time are valid.

Parameters:

`date` - the date to combine with, not null

Returns:

the local date-time formed from this time and the specified date, not null

atOffset

```
public OffsetTime atOffset(ZoneOffset offset)
```

Combines this time with an offset to create an `OffsetTime`.

This returns an `OffsetTime` formed from this time at the specified offset. All possible combinations of time and offset are valid.

Parameters:

`offset` - the offset to combine with, not null

Returns:

the offset time formed from this time and the specified offset, not null

toSecondOfDay

```
public int toSecondOfDay()
```

Extracts the time as seconds of day, from `0` to `24 * 60 * 60 - 1`.

Returns:

the second-of-day equivalent to this time

toNanoOfDay

```
public long toNanoOfDay()
```

Extracts the time as nanos of day, from `0` to `24 * 60 * 60 * 1,000,000,000 - 1`.

Returns:

the nano of day equivalent to this time

toEpochSecond

```
public long toEpochSecond(LocalDate date,
                          ZoneOffset offset)
```

Converts this `LocalTime` to the number of seconds since the epoch of 1970-01-01T00:00:00Z.

This combines this local time with the specified date and offset to calculate the epoch-second value, which is the number of elapsed seconds from 1970-01-01T00:00:00Z. Instants on the time-line after the epoch are positive, earlier are negative.

Parameters:

`date` - the local date, not null

`offset` - the zone offset, not null

Returns:

the number of seconds since the epoch of 1970-01-01T00:00:00Z, may be negative

Since:

9

compareTo

```
public int compareTo(LocalTime other)
```

Compares this time to another time.

The comparison is based on the time-line position of the local times within a day. It is "consistent with equals", as defined by `Comparable`.

Specified by:

`compareTo` in interface `Comparable`<`LocalTime`>

Parameters:

`other` - the other time to compare to, not null

Returns:

the comparator value, that is less than zero if this is before `other`, zero if they are equal, or greater than zero if this is after `other`

See Also:

`isBefore(LocalTime)`, `isAfter(LocalTime)`

isAfter

```
public boolean isAfter(LocalTime other)
```

Checks if this time is after the specified time.

The comparison is based on the time-line position of the time within a day.

Parameters:

`other` - the other time to compare to, not null

Returns:

true if this is after the specified time

isBefore

```
public boolean isBefore(LocalTime other)
```

Checks if this time is before the specified time.

The comparison is based on the time-line position of the time within a day.

Parameters:

`other` - the other time to compare to, not null

Returns:

true if this point is before the specified time

equals

```
public boolean equals(Object obj)
```

Checks if this time is equal to another time.

The comparison is based on the time-line position of the time within a day.

Only objects of type `LocalTime` are compared, other types return false. To compare the date of two `TemporalAccessor` instances, use `ChronoField.NANO_OF_DAY` as a comparator.

Overrides:

`equals` in class `Object`

Parameters:

`obj` - the object to check, null returns false

Returns:

true if this is equal to the other time

See Also:

`Object.hashCode()`, `HashMap`

hashCode

```
public int hashCode()
```

A hash code for this time.

Overrides:

`hashCode` in class `Object`

Returns:

a suitable hash code

See Also:

`Object.equals(java.lang.Object)`,
`System.identityHashCode(Object)`

toString

```
public String toString()
```

Outputs this time as a `String`, such as `10:15`.

The output will be one of the following ISO-8601 formats:

- `HH:mm`
- `HH:mm:ss`
- `HH:mm:ss.SSS`
- `HH:mm:ss.SSSSSS`
- `HH:mm:ss.SSSSSSSSS`

The format used will be the shortest that outputs the full value of the time where the omitted parts are implied to be zero.

Overrides:

`toString` in class `Object`

Returns:

a string representation of this time, not null

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.